

Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



- suspend function
- scope
- builders
- exceptions
- flow

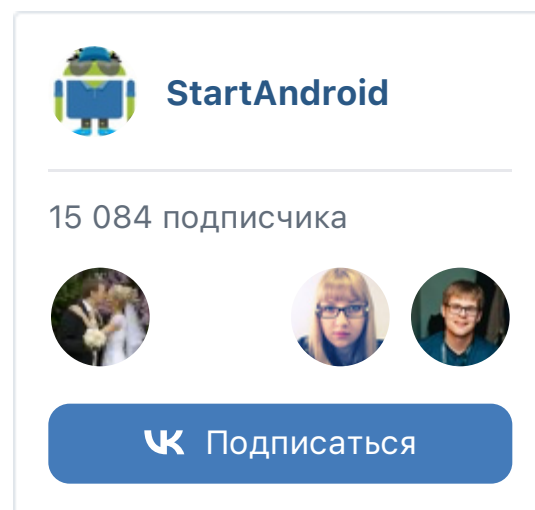
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

[Яндекс](#)
410011180491924
[WebMoney](#)
R248743991365
Z551306702056
[Paypal](#)

Урок 17. Hilt. Модули. Scope.

03 сентября 2021

В этом уроке разбираемся, как подключать модули и использовать Scope.

При использовании даггера мы в компоненте указываем scope и даггер-модули. Но Hilt создаёт компоненты сам, под капотом. Мы не имеем к ним доступ. А значит, не можем добавлять туда модули или scope тем же путем, каким привыкли это делать в даггере.

Давайте смотреть, как это делается в случае Hilt.

Модули

Для примера рассмотрим такой объект:

```
1 class NetworkUtils() {
2
3 }
```

Пусть он создается в модуле, а не Inject конструктором:

```
1 @Module
2 class NetworkModule {
3
4     @Provides
5     fun provideNetworkUtils(): NetworkUtils {
6         return NetworkUtils()
7     }
8
9 }
```

В даггере мы в компоненте указывали список используемых модулей. А в Hilt надо делать наоборот. В модуле указываем, в каком компоненте он будет использоваться.

Для этого используется аннотация InstallIn:

```
1 @Module
2 @InstallIn(SingletonComponent::class)
3 class NetworkModule {
4
5     @Provides
6     fun provideNetworkUtils(): NetworkUtils {
7         return NetworkUtils()
8     }
9
10 }
```

SingletonComponent тут означает AppComponent, который Hilt создаёт под капотом.

Т.е. наш модуль будет использован в AppComponent. А значит, AppComponent теперь знает, как создать объект NetworkUtils и сможет нам его предоставить:

```
1 @HiltAndroidApp
2 class App: Application() {
3
4     @Inject lateinit var networkUtils: NetworkUtils
5
6     override fun onCreate() {
7         super.onCreate()
8         Log.d(TAG, "NetworkUtils $networkUtils")
9     }
10
11 }
```

Давайте сразу вспомним уроки про сабкомпоненты. В них мы обсуждали, что если компонент умеет создавать объект, то и все его сабкомпоненты умеют. В случае с Hilt у нас создается такая иерархия компонентов:

AppComponent > ActivityComponent > FragmentComponent

Это приводит к тому, что если AppComponent умеет создавать объект, то и ActivityComponent и FragmentComponent также будут это уметь.

Теперь давайте в модуле укажем, что он будет использоваться не в AppComponent, а в ActivityComponent:

```
1 @Module
2 @InstallIn(ActivityComponent::class)
3 class NetworkModule {
4
5     @Provides
6     fun provideNetworkUtils(): NetworkUtils {
7         return NetworkUtils()
8     }
9
10 }
```

Для этого в аннотации InstallIn указываем класс ActivityComponent. Теперь ActivityComponent умеет сам создавать NetworkUtils.

Снова посмотрим на иерархию компонентов:

AppComponent > ActivityComponent > FragmentComponent

FragmentComponent также будет уметь создавать объект NetworkUtils, т.к. он является сабкомпонентом для ActivityComponent.

А вот AppComponent уже ничего не будет знать про этот объект и не сможет его создать.

Если мы хотим еще больше сузить возможность создания объекта и использовать модуль в FragmentComponent, то в InstallIn указываем FragmentComponent.

Полный список компонентов можно посмотреть в [документации](#). Мы пока знаем только три из них, самые понятные и очевидные. Так проще вникать в тему. Все остальные мы обсудим в следующем уроке.

Доступные объекты

Компоненты в Hilt по умолчанию умеют предоставлять Android объекты, которые могут нам понадобиться. Например, AppComponent может предоставить объект Application, а ActivityComponent - объект Activity.

Т.е. при создании экземпляра компонента ActivityComponent для очередного Activity, Hilt передает компоненту это Activity. А компонент уже может предоставить его нам напрямую или использовать при создании других объектов.

Мы, кстати, делали подобные вещи с помощью кастомных билдеров и фабрик. Создавали компоненты и передавали туда Activity, чтобы компонент мог его использовать для создания объектов.

Если компоненты могут предоставлять нам Android объекты, то мы можем в своих модулях их получать и использовать. Например, если мы указываем, что модуль используется в AppComponent (@InstallIn(SingletonComponent::class)), то мы точно знаем, что в этом модуле можно запросить объект Application в Provides методе:

```
1 @Module
2 @InstallIn(SingletonComponent::class)
3 class NetworkModule {
4
5     @Provides
6     fun provideNetworkUtils(app: Application): NetworkUtils {
7         return NetworkUtils(app)
8     }
9
10 }
```

AppComponent содержит этот объект и сможет нам его предоставить.

Если модуль используется в ActivityComponent, то в нем можно запросить объект Activity. А также нам доступен и Application объект. Потому что сабкомпонентам (ActivityComponent) доступны объекты их родительских компонентов (AppComponent):

```
1 @Module
2 @InstallIn(ActivityComponent::class)
3 class NetworkModule {
4
5     @Provides
6     fun provideNetworkUtils(app: Application, activity: Activity): NetworkUtils {
7         return NetworkUtils(...)
8     }
9
10 }
```

Ну и, соответственно, если используем модуль в FragmentComponent, то нам доступны и фрагмент, и Activity, и Application:

```
1 @Module
2 @InstallIn(FragmentComponent::class)
3 class NetworkModule {
4
5     @Provides
6     fun provideNetworkUtils(app: Application, activity: Activity, fragment: Fragment): Network
7         return NetworkUtils(...)
8     }
9
10 }
```

Полный список доступных каждому компоненту объектов можно посмотреть в [документации](#).

Scope

Как мы помним из прошлых уроков, Scope аннотация вешается на компонент. А потом уже мы используем ее с объектами, чтобы компонент создавал синглтоны этих объектов.

У нас нет доступа к компонентам, которые создает Hilt. Мы не можем повесить на них Scope аннотации. Но Hilt сделал это за нас.

Мы знаем пока три компонента: AppComponent, ActivityComponent и FragmentComponent. Для каждого из них Hilt создал соответствующую Scope аннотацию: Singleton, ActivityScoped, и FragmentScoped.

Нам остается просто повесить нужную аннотацию на наш объект, как мы делали ранее с даггером:

```
1 @ActivityScoped
2 class NetworkUtils @Inject constructor() {
3
4 }
```

ActivityComponent теперь будет создавать синглтон этого объекта.

Напомним, что для каждого Activity создается свой экземпляр ActivityComponent. И каждый такой экземпляр компонента создаёт свой синглтон объекта NetworkUtils и предоставляет его в Activity. Т.е. у каждого Activity будет свой синглтон. И он же будет использован во фрагментах этого Activity, т.к. FragmentComponent будет брать его из родительского ActivityComponent.

Если же нам надо, чтобы все Activity получали один и тот же синглтон, то вешаем на объект аннотацию компонента AppComponent, т.е. - Singleton. В этом случае синглтон объекта будет создан в AppComponent. И все экземпляры ActivityComponent будут иметь к нему доступ.

Полный список Scope аннотаций можно посмотреть в [документации](#).

Назад

Вперёд

Комментарии

Крутяк! — Владимир 04.09.2021 18:43
С нетерпением жду продолжения)

Ответить

Singletons — Rost 11.09.2021 10:54
В 14 уроке было сказано, что активности и фрагмент должны иметь один и тот же экземпляр сабкомпонента, чтобы получить одинаковый синглтон. Как будет в случае с хилт? Разве фрагмент не получит новый экземпляр сабкомпонента(FragmentComponent), отличный от ActivityComponent с новым синглтоном внутри?

Ответить

RE: Singletons — Dmitry Vinogradov 18.09.2021 17:25
Смотря где живет синглтон.

Если в FragmentComponent, то каждый фрагмент получит свой экземпляр этого синглтона, т.к. у каждого фрагмента будет свой экземпляр FragmentComponent.

А если синглтон живет в ActivityComponent, то все фрагменты этого Activity получат именно этот синглтон. Потому что все экземпляры сабкомпонента FragmentComponent в этих фрагментах будут брать синглтон из одного общего ActivityComponent. Сами они его создавать не будут

Ответить

🔄 Обновить список комментариев

📡 RSS лента комментариев этой записи

Добавить комментарий

Тема

😄 😊 😍 😂 😇 😈 😉 😋 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿

Отправить

3Comments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора
При возникновении каких-либо вопросов, используйте [форму обратной связи](#)

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under [MIT License](#).
Font Awesome font licensed under [SIL OFL 1.1](#).